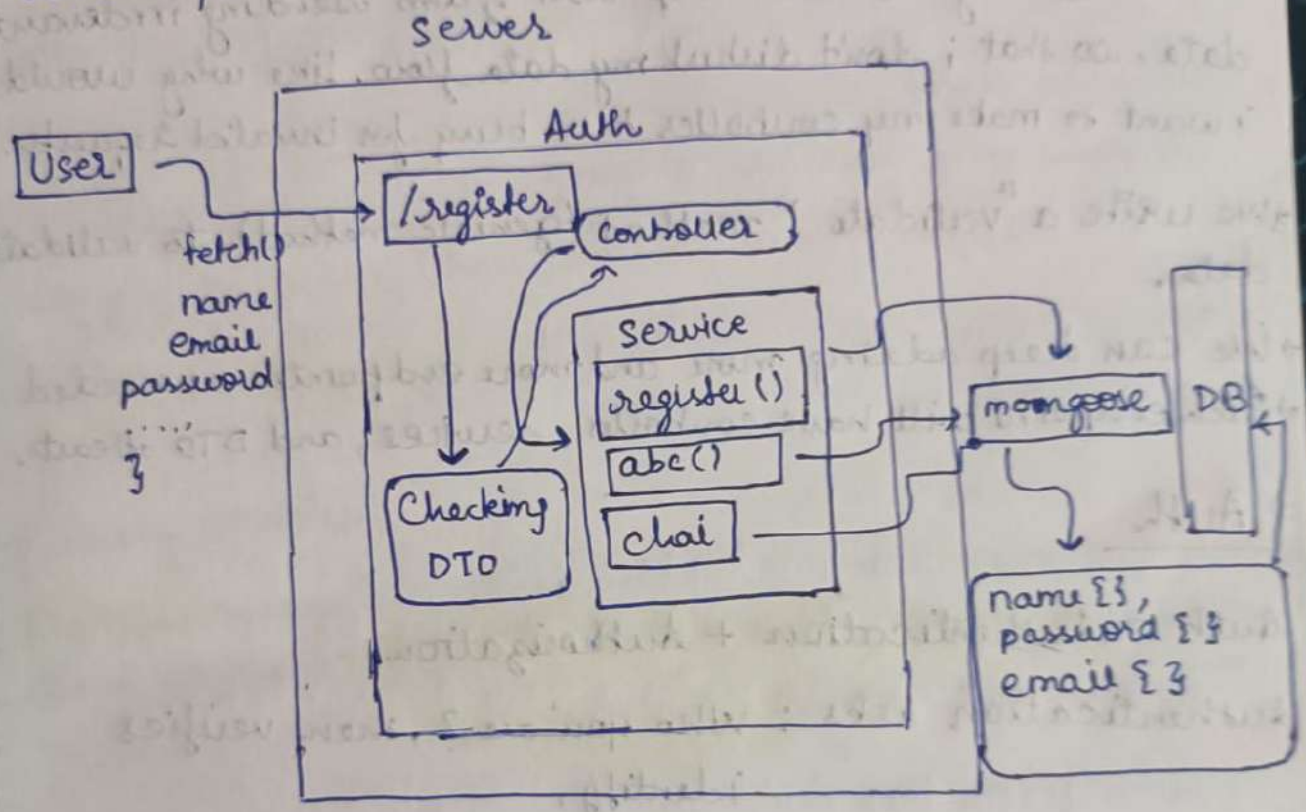
BUILDING RESTful API (Part 2)

Note: Beyond any good practices, the business model comes first.

User workflow:

common folder contains basic template for common services



Mongoose takes care of the shape of the schema stored in the DB. When we create a schema, mongoose enforces that data must be stored in the shape, as we defined.

On the /register endpoint, we have some functionalities on it
eg register () → this is just a service

Controller's work is to call service, whichever needed. All services have access to mongoose

* As soon as we hit on /register, we must do validation or checking, as if we do it after that (its costly and inefficient) as data flows from controller → service → mongoose → DB

2

⇒ So it always better to stop user from sending irrelevant data, so that i don't disturb my data flow. Like why would i want to make my controller to be busy for invalid requests.

⇒ We write a "validate" method (generic method) to validate data.

⇒ We can keep adding more and more endpoints as needed

⇒ Each endpoint will have controller, services, and DTO atleast.

⇒ **Auth**

Auth = Authentication + Authorization

Authentication asks : Who you are ?, then verifies identity,

If success : I know who you are

If fail : I don't know who u are

Authorization asks : What are you allowed to do?

Happens after authentication

Controls access

eg:

⇒ After a user login's, make a entry in DB, and store {session : true, id}

⇒ So whenever you asks for a service, a db call is made check if {session : true} then gives access accordingly
if {session : false} then deny access.

⇒ Session is also called Stateful

A better authentication mechanism than session.
In this we generate a long string and giver user a copy of the string (as a cookies / token etc).

Cookies are of two types
- Normal cookies
- Secure cookies.

Stateless means the token / cookie is valid for a particular time. Stateless = server does not remember the user session.
This token is kept in a format : Bearer "long string"
↓
whoever holds that.

So whoever has the string is the owner.
In stateless, its the responsibility of frontend to send the token as using cryptography we check the valid token.
Token can be made by various methods :
1. Random string
2. Token : XXXX.YYYY.ZZZZ (Header.Payload.Signature)
(Signature must be kept safe always)

Payload: The payload contains user data.

eg: { "userId": "123",
"role": "admin"
}

Header: tells us how the token has been created.

eg:
{
"alg": "HS256", ← [signing algorithm]
"typ": "JWT" ← [token type]
}

⇒ Header defines the algo, payload contains the user data, and signature ensures the token hasn't been tampered.

⇒ **Access Token** is short lived, and sent to user, not stored in db.
Refresh Token is long lived and stored in db.

For eg: If your access token expires, we send a particular 206 error (when you try to access using expired token) and the db identifies the error, checks if refresh Token is still valid and then generates a new Access token.

⇒ Frontend also has middleware/interceptors, which stops 206 error to be sent to user, and retries the request if refresh token is still valid.

All this depends on business use case, so when new access token is generated, we generally generate a new refresh token also. ~~which~~

Note: In some stateless Implementation, refresh token is not stored in db, just verified using the signature.

In our .env file

ADD : JWT_ACCESS_SECRET = alongstring

JWT_ACCESS_EXPIRES_IN = 15m

JWT_REFRESH_TOKEN = somethingsecret

JWT_REFRESH_EXPIRES_IN = 7d

5

Encrypt and hash are used interchangebly.

Encrypt can be decrypted

Hash cannot be reversed, one-way, used generally for passwords. The password is hashed using salt

encrypt ⇒ 123 ⇒ abc

hash ⇒ 123 ⇒ 12243dwtgv dfg.

but if your secret is same, so using salt, we can verify if the password is same.

→ npm i jsonwebtoken

⇒ "inside jwt.utils.js

```
import jwt from "jsonwebtoken"

const generateAccessToken = (payload) => {
    jwt.sign(payload, process.env.JWT_ACCESS_SECRET, {
    expiresIn : process.env.JWT_ACCESS_EXPIRES_IN || '15m'
} })

// ~~const~~ fun' to verify the access token

const verifyAccessToken = (token) => {
return jwt.verify (token, process.env.JWT_ACCESS_~~TOKEN~~ SECRET)
}
```

gives T/F

```
// generate RefreshToken
const generateRefreshToken = (payload) => {
  return jwt.sign(payload, process.env.JWT_REFRESH_SECRET, {
    expiresIn : process.env.JWT_REFRESH_EXPIRES_IN || "7d"
  })
};

// verify Refresh Token
const verifyRefreshToken = (token) => {
  return jwt.verify(token, process.env.JWT_REFRESH_SECRET)
}
```

⇒ Now in our auth.service.js
Make new service for login

```
const login = async ({email, password}) => {
  // take email and find user in DB
  // then check if password is correct
  // check if verified or not
const user = await User.findOne({email}).select("+password")
if (!user) throw ApiError.unauthorized("Invalid email or password")

// somehow we we check pass and also update errors
in our api-error.js file
  if (!user.isVerified) {
    throw ApiError.forbidden("Pls verify email before login");
}
```

custom Error

⇒ Now lets generate the access and refresh token ⇒
code continued ⇒
~~generateAccessToken({ id : user_id~~

7

```
const accessToken = generateAccessToken({ id : user._id ,
                                          role : user.role });

const refreshToken = generateRefreshToken({ id : user._id })

//Now we update the refreshToken,
  user.refreshToken = hashToken( refreshToken);
```

Now we need to store the refreshToken in DB, but we first need to hash it.

```
// Make hashToken
  const hashToken = (token) ⇒ crypto.createHash("sha256")
                                .update(rawToken)
                                .digest("hex");

// Now store it in db, also remember db always in another continent
  await user.save({ validateBeforeSave : false })

// Now lets makes the user Object
  const userObject = user.toObject()
  delete userObj.password
  delete userObj.refreshToken
  return { user : userObj, accessToken, refreshToken }
```

} deleted from object not from db

? ⇒ Always question stuff

Note: Mobile apps do not have cookies, so we need to support mobile apps also, so above method is preferred in this case.

8

+ If you want to send cookies instead, use cookie-parser

Assignment: Try sending tokens in form of cookies.

// Now let's write a refresh service, which will be used to refresh Access token

refreshToken (as we reach here when Access token is expired)

```
const refresh = async(token) => {
    if (!token) throw ApiError.unauthorized("Refresh token missing")
const decoded = verifyRefreshToken(token)
```

will save you from vulnerabilities (verify is done using signature)

// still with above line can't verify if the token belong to the same user who sent the token, so need to verify even further.

has same payload

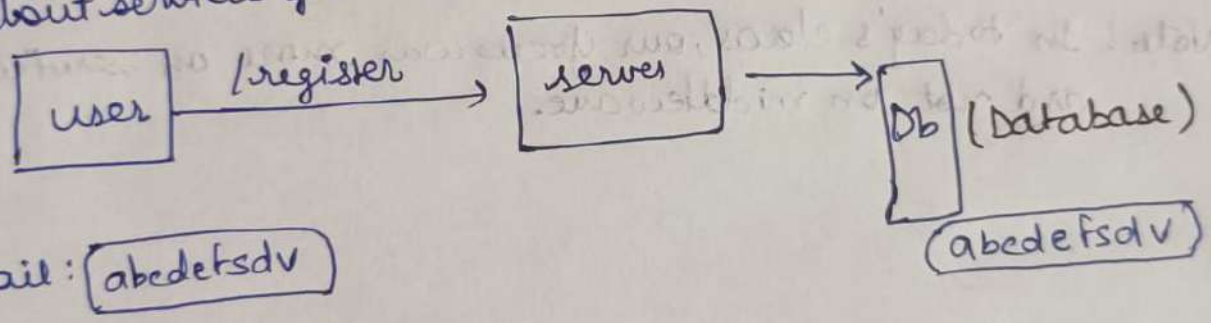
```
const user =
await User.findById(decoded.id).select("+refreshToken");
    if(!user) throw ApiError.unauthorized("User Not found")
        if(User.refreshToken !== hashToken(token)){
            throwApiError.unauthorized("Invalid refresh token");
    }
const accessToken = generateAccessToken({id: user._id, role: user.role})
        return{accessToken};
```

Now we can make a endpoint for refresh that.
If you want to logout just we need to expire refresh Token.

```
const logout = async (userId) => {
    const user = await User.findById(userId);
    if (!user) throw ApiError.unauthorized("User not found")
    user.refreshToken = undefined
    await user.save({ validateBeforeSave: false });
    await User.findByIdAndUpdate(userId, { refreshToken: null })
}
```

OTP and Token sending mechanism are almost same.

Suppose we have a endpoint, to verify user. We only care about services for now:



mail: abcdefsdv

/forgot-password → mail: jaani@gmail.com
take mail from user
verify if email exists in db
create Token
send Token to user and
store in DB

/new-password ⇒ take token from user
verify from DB
take new password from user
update it in new password in DB

```
const forgotPassword = async(email) => {
    const user = await User.findOne({email})
    if (!user) throw ApiError: notfound | "No account with
                                          that email");

    const {rawToken, hashedToken} = generateResetToken()
    user.resetPasswordToken = hashedToken
    user.resetPasswordExpires = Date.now()+15*60*1000
    await user.save()

    // TODO : mail bhejna nhi aata
// Assignment : Write service for new-password/reset Password
```

Note: In today's class, our focus was more on services and not on middleware.